

Enterprise Network Automation and Hybrid-Cloud Infrastructure as Code

Project ENT-HYBRID-NET: GNS3, Ansible, Git and Terraform on AWS

Item	Details
Unit	[Unit code and name]
Group	[Group name / number]
Members	[Student name - ID] [Student name - ID] [Student name - ID]
Lecturer	[Name]
Git repository	https://github.com/shakil-shahan02/Enterprise-Network-Automation
Date	[dd/mm/yyyy]

Table of Contents

1. Executive Summary	3
2. Introduction	4
3. Enterprise Network Design	5
3.1 Topology.....	5
3.2 IP addressing	5
3.3 Configuration and justification	6
3.4 Connectivity verification	6
4. Network Automation with Ansible	11
4.1 Day-2 change through Git	14
4.2 Infrastructure as Code discussion	15
5. AWS Infrastructure as Code with Terraform	16
6. Testing.....	20
7. Discussion.....	21
8. Conclusion	22
References	23
Appendix: Submission Contents	24

1. Executive Summary

A multinational enterprise engaged our team to replace manual, device-by-device administration with an automated, secure and repeatable hybrid-cloud platform. We delivered three outcomes. First, a hierarchical GNS3 network (ENT-HYBRID-NET) connecting Headquarters, a Branch Office and a Data Centre with 4 routers, 6 switches and a Linux automation server, providing VLANs, inter-VLAN routing, multi-area OSPF, DHCP, DNS, NTP, Syslog, SSH, ACLs and NAT. Second, an Ansible project that treats the network as code: YAML data and Jinja2 templates drive eight roles that back up, configure, harden, verify and audit all ten devices idempotently, with credentials protected by Ansible Vault. Third, Terraform code that provisions a secure two-AZ AWS VPC with a bastion host and a private EC2 instance; it was deployed, verified, destroyed and audited to confirm no ongoing cost. All work is version-controlled in Git.

Testing confirmed the design. Every OSPF adjacency reached the FULL state, hosts received addresses from DHCP and resolved internal and Internet names, and each security policy behaved as intended. The Ansible pipeline passed all verification checks, scored fourteen out of fourteen in the compliance audit on every device, and reported no changes on a second run. The AWS environment was built, reached through the bastion host and then removed without leaving any billable resources.

2. Introduction

Manual CLI administration causes inconsistent configurations, slow changes and weak audit trails. The objectives were to design a scalable and secure enterprise network (Part A), automate its configuration and validation with Ansible (Part B), and provision cloud infrastructure with Terraform stored in Git (Part C).

The scope covered three on-premises sites and one AWS region. Headquarters hosts the IT, Finance and Guest departments; the Branch Office hosts Sales and Operations; the Data Centre hosts servers and the network-operations (NetOps) tools. The same engineering principles were applied throughout: define the desired state as code, keep it in Git, apply it automatically, and prove the result with repeatable tests. This report describes the design, the automation, the cloud deployment and the testing, then reflects on the challenges and on future improvements.

Component	Version	Role
GNS3	2.2.61 + GNS3 VM	Network emulation; topology built through its REST API
Routers	Cisco 7200, IOS 15.0(1)M	R1-HQ, R2-BR, R3-DC, R4-CORE
Switches	Cisco 2691 + NM-16ESW, IOS 12.4	Three distribution and three access switches
Automation server	Debian container, ansible-core 2.17, cisco.ios 11.6	Ansible control node and Syslog collector
Terraform	1.16.2, AWS provider 6.66	AWS Infrastructure as Code
Git / GitHub	2.54	Version control for all code and evidence

Table 1: Tools and versions

3. Enterprise Network Design

3.1 Topology

The network follows the three-layer hierarchical model (Figure 1). R4-CORE forms the core and Internet edge. The site routers R1-HQ, R2-BR and R3-DC, together with the distribution switches, form the distribution layer. Access switches connect end hosts. Each site is a separate block, so a new site can be attached to the core without redesign. The lab was created by a Python script using the GNS3 REST API, making the topology itself reproducible. End hosts are lightweight Linux containers, which provide real tools such as nslookup, wget and traceroute for testing. The Data Centre also contains an nginx web server so that the server-protection ACL could be tested with genuine HTTP traffic.

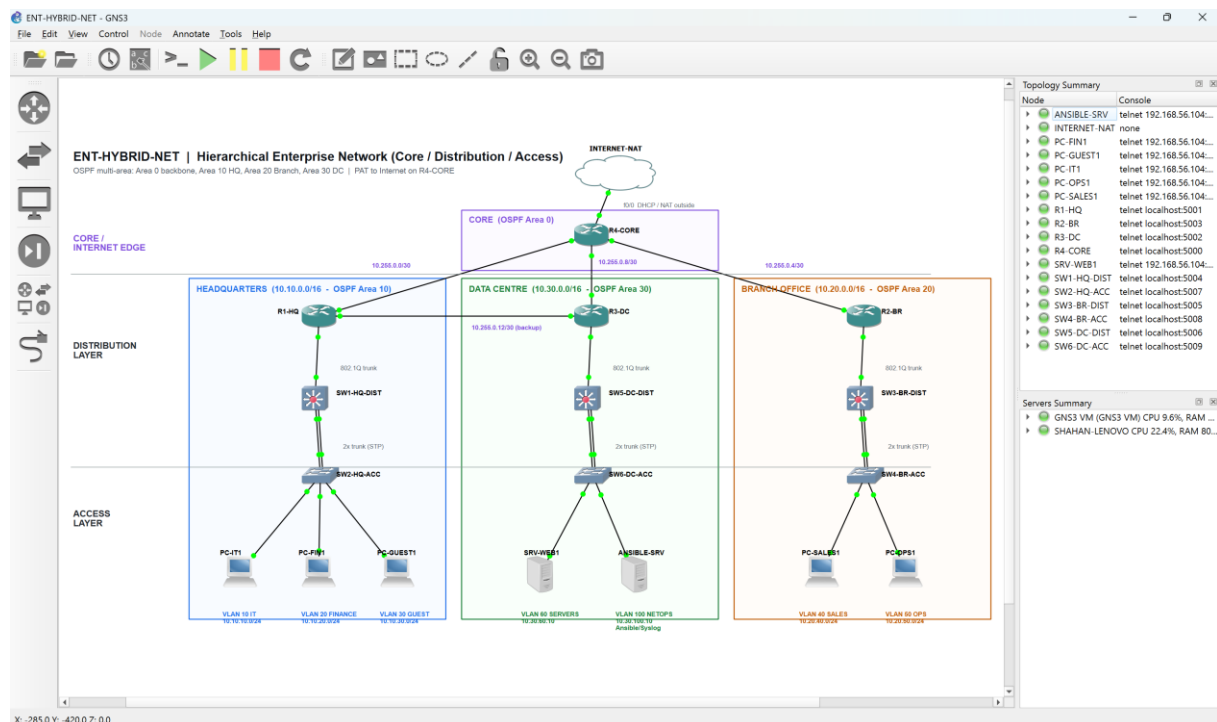


Figure 1: GNS3 topology: core, distribution and access layers across three sites

3.2 IP addressing

Each site owns a summarisable /16, loopbacks use 10.0.0.0/24 and point-to-point links use /30s from 10.255.0.0/24. The AWS VPC uses 172.20.0.0/16 so it never overlaps on-premises ranges.

Site	VLAN	Subnet	Gateway	Notes
HQ	10 IT	10.10.10.0/24	R1 Fa0/0.10	DHCP; may manage devices
HQ	20 Finance	10.10.20.0/24	R1 Fa0/0.20	DHCP; FINANCE-PROTECT ACL
HQ	30 Guest	10.10.30.0/24	R1 Fa0/0.30	DHCP; Internet-only ACL
Branch	40 Sales / 50 Ops	10.20.40-50.0/24	R2 sub-interfaces	DHCP
DC	60 Servers	10.30.60.0/24	R3 Fa0/0.60	Web server 10.30.60.10
DC	100 NetOps	10.30.100.0/24	R3 Fa0/0.100	Ansible and Syslog 10.30.100.10
All	99 Mgmt	10.x.0.99.0/24	Site router	Switch management SVIs
Core	-	10.255.0.0/24 (/30s)	-	R4 to R1, R2, R3; R1-R3 backup

Table 2: VLAN and subnet plan

3.3 Configuration and justification

- **VLANs and inter-VLAN routing:** department VLANs isolate broadcast domains; router-on-a-stick sub-interfaces give one policy point per site. Dual trunks with a fixed STP root provide loop-free redundancy.
- **OSPF:** multi-area design with area 0 on the backbone and one area per site. ABRs summarise each site to a single /16, area 0 uses MD5 authentication, and the R1-R3 link (cost 100) is a backup path.
- **Services:** per-site DHCP keeps branches working during WAN failure; R3-DC is the corp.local DNS server; R4 is the NTP master; every device logs to the NetOps Syslog collector.
- **Security:** SSH v2 only with an ACL restricting management to the NetOps and IT VLANs; ACLs keep guests Internet-only, isolate Finance and protect servers; PAT on R4 provides a single controlled Internet exit.
- **Why this design:** the hierarchical model keeps failure domains small and makes troubleshooting predictable, because each layer has a single job. OSPF is an open standard that converges quickly and scales through areas, so the core routing table holds only three site summaries instead of every subnet. Local DHCP and a central DNS server balance resilience with consistency. Placing the automation server in the Data Centre NetOps VLAN, the only network allowed to manage devices apart from HQ IT, follows the principle of least privilege for the management plane.

Each service was configured with a clear purpose. DHCP pools on R1 and R2 exclude the first twenty addresses of every subnet for gateways and infrastructure, and hand out the default gateway, the corp.local domain and the DNS server address. R3-DC answers DNS queries for internal host names and forwards other queries to a public resolver through the NAT edge. R4-CORE acts as the NTP master for the whole enterprise, and every device sends informational Syslog messages, sourced from its loopback or management interface, to the collector on ANSIBLE-SRV. On the switches, unused ports are shut down and access ports use PortFast so hosts connect quickly without risking loops.

3.4 Connectivity verification

```

Part A | R4-CORE | OSPF adjac
=====
Part A | R4-CORE | OSPF adjacencies + routing table [localhost:5000] 2026-09-25 14:47:26
=====

R4-CORE#show ip ospf neighbor

Neighbor ID      Pri   State           Dead Time   Address        Interface
10.0.0.3         0     FULL/-          00:00:32    10.255.0.10    FastEthernet2/0
10.0.0.2         0     FULL/-          00:00:34    10.255.0.6     FastEthernet1/1
10.0.0.1         0     FULL/-          00:00:33    10.255.0.2     FastEthernet1/0

R4-CORE#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, + - replicated route

Gateway of last resort is 192.168.42.1 to network 0.0.0.0

S*    0.0.0.0/0 [254/0] via 192.168.42.1
      10.0.0.0/8 is variably subnetted, 14 subnets, 3 masks
O      10.0.0.1/32 [110/11] via 10.255.0.2, 00:01:56, FastEthernet1/0
O      10.0.0.2/32 [110/11] via 10.255.0.6, 00:22:57, FastEthernet1/1
O      10.0.0.3/32 [110/11] via 10.255.0.10, 00:01:09, FastEthernet2/0
C      10.0.0.4/32 is directly connected, Loopback0
O IA   10.10.0.0/16 [110/20] via 10.255.0.2, 00:01:56, FastEthernet1/0
O IA   10.20.0.0/16 [110/20] via 10.255.0.6, 00:22:57, FastEthernet1/1
O IA   10.30.0.0/16 [110/20] via 10.255.0.10, 00:01:09, FastEthernet2/0
C      10.255.0.0/30 is directly connected, FastEthernet1/0
L      10.255.0.1/32 is directly connected, FastEthernet1/0
C      10.255.0.4/30 is directly connected, FastEthernet1/1
L      10.255.0.5/32 is directly connected, FastEthernet1/1
C      10.255.0.8/30 is directly connected, FastEthernet2/0
L      10.255.0.9/32 is directly connected, FastEthernet2/0
O      10.255.0.12/30 [110/110] via 10.255.0.10, 00:01:09, FastEthernet2/0
      [110/110] via 10.255.0.2, 00:01:56, FastEthernet1/0
      192.168.42.0/24 is variably subnetted, 2 subnets, 2 masks
C      192.168.42.0/24 is directly connected, FastEthernet0/0
L      192.168.42.241/32 is directly connected, FastEthernet0/0

R4-CORE#
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation>

```

Figure 2: R4-CORE: three FULL OSPF neighbours, site summaries and default route

```

Part A | SW1-HQ-DIST | VLAN
=====
Part A | SW1-HQ-DIST | VLAN database + 802.1Q trunks [localhost:5004] 2026-09-25 14:49:32
=====

SW1-HQ-DIST#show vlan-switch brief

VLAN Name                Status    Ports
-----
1    default                active    Fa1/3, Fa1/4, Fa1/5, Fa1/6, Fa1/7, Fa1/8, Fa1/9, Fa1/10, Fa1/11, Fa1/12, Fa1/13, Fa1/14, Fa1/15
10   IT                       active
20   FINANCE                 active
30   GUEST                   active
99   MGMT                     active
1002 fddi-default            active
1003 token-ring-default    active
1004 fddinet-default        active
1005 trnet-default          active

SW1-HQ-DIST#show interfaces trunk

Port      Mode        Encapsulation  Status      Native vlan
Fa1/0     on          802.1q         trunking    1
Fa1/1     on          802.1q         trunking    1
Fa1/2     on          802.1q         trunking    1

Port      Vlans allowed on trunk
Fa1/0     1-1005
Fa1/1     1-1005
Fa1/2     1-1005

Port      Vlans allowed and active in management domain
Fa1/0     1,10,20,30,99
Fa1/1     1,10,20,30,99
Fa1/2     1,10,20,30,99

Port      Vlans in spanning tree forwarding state and not pruned
Fa1/0     1,10,20,30,99
Fa1/1     1,10,20,30,99
Fa1/2     1,10,20,30,99

SW1-HQ-DIST#
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation>

```

Figure 3: SW1-HQ-DIST: VLAN database and 802.1Q trunks

```

Part A | SW2-HQ-ACC | Spann  X  +  v

Port path cost 19, Port priority 128, Port Identifier 128.42.
Designated root has priority 4096, address c005.189c.0001
Designated bridge has priority 4096, address c005.189c.0001
Designated port id is 128.42, designated path cost 0
Timers: message age 1, forward delay 0, hold 0
Number of transitions to forwarding state: 1
BPDU: sent 8, received 906

Port 43 (FastEthernet1/2) of VLAN10 is blocking
Port path cost 19, Port priority 128, Port Identifier 128.43.
Designated root has priority 4096, address c005.189c.0001
Designated bridge has priority 4096, address c005.189c.0001
Designated port id is 128.43, designated path cost 0
Timers: message age 1, forward delay 0, hold 0
Number of transitions to forwarding state: 0
BPDU: sent 7, received 906

Port 46 (FastEthernet1/5) of VLAN10 is forwarding
Port path cost 19, Port priority 128, Port Identifier 128.46.
Designated root has priority 4096, address c005.189c.0001
Designated bridge has priority 32768, address c008.0c94.0001
Designated port id is 128.46, designated path cost 19
Timers: message age 0, forward delay 0, hold 0
Number of transitions to forwarding state: 1
BPDU: sent 910, received 0
The port is in the portfast mode

SW2-HQ-ACC#show interfaces status

Port      Name                Status      Vlan      Duplex Speed Type
Fa1/0     notconnect         1           auto      auto   10/100BaseTX
Fa1/1     TRUNK to SW1-HQ-DI connected     trunk     a-full   a-100   10/100BaseTX
Fa1/2     TRUNK to SW1-HQ-DI connected     trunk     a-full   a-100   10/100BaseTX
Fa1/3     UNUSED             disabled     1         auto      auto   10/100BaseTX
Fa1/4     UNUSED             disabled     1         auto      auto   10/100BaseTX
Fa1/5     ACCESS PC-IT1 (VLA connected     10        a-full   a-100   10/100BaseTX
Fa1/6     ACCESS PC-FIN1 (VL connected     20        a-full   a-100   10/100BaseTX
Fa1/7     ACCESS PC-GUEST1 ( connected     30        a-full   a-100   10/100BaseTX
Fa1/8     UNUSED             disabled     1         auto      auto   10/100BaseTX
Fa1/9     UNUSED             disabled     1         auto      auto   10/100BaseTX
Fa1/10    UNUSED             disabled     1         auto      auto   10/100BaseTX
Fa1/11    UNUSED             disabled     1         auto      auto   10/100BaseTX
Fa1/12    UNUSED             disabled     1         auto      auto   10/100BaseTX
Fa1/13    UNUSED             disabled     1         auto      auto   10/100BaseTX
Fa1/14    UNUSED             disabled     1         auto      auto   10/100BaseTX
Fa1/15    UNUSED             disabled     1         auto      auto   10/100BaseTX
SW2-HQ-ACC#
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation>

```

Figure 4: SW2-HQ-ACC: redundant uplink blocked by spanning tree

```

Part A | PC-IT1 (HQ VLAN 10) | DHCP lease, DNS, inter-site + Internet connectivity [192.168.56.104:5015] 2026-09-25 14:52:54

PC-IT1:/#ip -4 addr show eth0 | grep inet
    inet 10.10.10.24/24 scope global eth0
PC-IT1:/#cat /etc/resolv.conf
nameserver 10.0.0.3
PC-IT1:/#nslookup web.corp.local 2>/dev/null | grep -A1 Name
Name:    web.corp.local
Address: 10.30.60.10
PC-IT1:/#ping -c 2 -W 2 10.30.60.10
PING 10.30.60.10 (10.30.60.10): 56 data bytes
64 bytes from 10.30.60.10: seq=0 ttl=61 time=71.638 ms
64 bytes from 10.30.60.10: seq=1 ttl=61 time=71.278 ms

--- 10.30.60.10 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 71.278/71.458/71.638 ms
PC-IT1:/#ping -c 2 -W 2 10.20.40.22
PING 10.20.40.22 (10.20.40.22): 56 data bytes
64 bytes from 10.20.40.22: seq=0 ttl=61 time=129.971 ms
64 bytes from 10.20.40.22: seq=1 ttl=61 time=89.125 ms

--- 10.20.40.22 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 89.125/109.548/129.971 ms
PC-IT1:/#ping -c 2 -W 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=58 time=78.196 ms
64 bytes from 8.8.8.8: seq=1 ttl=58 time=65.113 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 65.113/71.654/78.196 ms
PC-IT1:/#traceroute -n -w 2 -q 1 10.20.40.22
traceroute to 10.20.40.22 (10.20.40.22), 30 hops max, 46 byte packets
 1  10.10.10.1  46.052 ms
 2  10.255.0.1  56.190 ms
 3  10.255.0.6  69.202 ms
 4  10.20.40.22  64.870 ms
PC-IT1:/#
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation>

```

Figure 5: PC-IT1: DHCP lease, DNS, inter-site and Internet reachability

```

Part A | PC-GUEST1 (HQ VLAN 30) | GUEST-IN ACL - Internet allowed, internal networks blocked [192.168.56.104:5005] 2026-09-25 15:03:31

PC-GUEST1:/#ip -4 addr show eth0 | grep inet
    inet 10.10.30.22/24 scope global eth0
PC-GUEST1:/#ping -c 2 -W 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=58 time=83.547 ms
64 bytes from 8.8.8.8: seq=1 ttl=58 time=55.480 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 55.480/69.513/83.547 ms
PC-GUEST1:/#nslookup google.com 2>/dev/null | grep -A1 Name
Name:    google.com
Address: 142.250.183.46
PC-GUEST1:/#ping -c 2 -W 2 10.30.60.10
PING 10.30.60.10 (10.30.60.10): 56 data bytes

--- 10.30.60.10 ping statistics ---
2 packets transmitted, 0 packets received, 100% packet loss
PC-GUEST1:/#ping -c 2 -W 2 10.10.20.23
PING 10.10.20.23 (10.10.20.23): 56 data bytes

--- 10.10.20.23 ping statistics ---
2 packets transmitted, 0 packets received, 100% packet loss
PC-GUEST1:/#
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation>

```

Figure 6: PC-GUEST1: Internet allowed, internal networks blocked

```

Part A | PC-SALES1 (Branch VLAN 40) | Web server reachable, Finance VLAN blocked by ACL [192.168.56.104:5007] 2026-09-25 15:05:32

PC-SALES1:/#ip -4 addr show eth0 | grep inet
inet 10.20.40.22/24 scope global eth0
PC-SALES1:/#nslookup web.corp.local 2>/dev/null | grep -A1 Name
Name: web.corp.local
Address: 10.30.60.10
PC-SALES1:/#wget -qO- http://10.30.60.10 | grep -i title
<title>Welcome to nginx!</title>
PC-SALES1:/#ping -c 2 -W 2 10.10.10.24
PING 10.10.10.24 (10.10.10.24): 56 data bytes
64 bytes from 10.10.10.24: seq=0 ttl=61 time=60.677 ms
64 bytes from 10.10.10.24: seq=1 ttl=61 time=62.987 ms

--- 10.10.10.24 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 60.677/61.832/62.987 ms
PC-SALES1:/#ping -c 2 -W 2 10.10.20.23
PING 10.10.20.23 (10.10.20.23): 56 data bytes

--- 10.10.20.23 ping statistics ---
2 packets transmitted, 0 packets received, 100% packet loss
PC-SALES1:/#traceroute -I -n -w 2 -q 1 10.30.60.10
traceroute to 10.30.60.10 (10.30.60.10), 30 hops max, 46 byte packets
 1 10.20.40.1 41.194 ms
 2 10.255.0.5 61.104 ms
 3 10.255.0.10 71.679 ms
 4 10.30.60.10 51.758 ms
PC-SALES1:/#
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation>

```

Figure 7: PC-SALES1: web server reachable, Finance VLAN blocked

```

LIVE: ANSIBLE SRV (Ansible: x + v
messages: 978 from 10 devices: 10.0.0.1 10.0.0.2 10.0.0.3 10.0.0.4 10.10.9.11 10.10.9.12 10.20.99.11 10.20.99.12 10.30.99.11 10.30.99.12

2026-09-25 19:06:51 10.20.99.12 NOTICE 81: .Sep 25 19:04:20.615 AEST: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: netadmin] [Source: 10.30.100.10] [localport
: 22] at 19:04:20 AEST Fr
2026-09-25 19:08:26 10.0.0.2 NOTICE 33: Sep 25 19:08:26.187 AEST: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: netadmin] [Source: 10.30.100.10] [localport:
22] at 19:08:26 AEST Fr
2026-09-25 19:10:55 10.0.0.2 NOTICE 34: Sep 25 19:10:53.675 AEST: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: netadmin] [Source: 10.30.100.10] [localport:
22] at 19:10:53 AEST Fr
2026-09-25 19:11:23 10.20.99.12 NOTICE 82: .Sep 25 19:08:43.879 AEST: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: netadmin] [Source: 10.30.100.10] [localport
: 22] at 19:08:43 AEST Fr
2026-09-25 19:11:23 10.20.99.11 NOTICE 82: .Sep 25 19:08:43.481 AEST: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: netadmin] [Source: 10.30.100.10] [localport
: 22] at 19:08:43 AEST Fr
2026-09-25 19:11:28 10.20.99.11 NOTICE 83: .Sep 25 19:08:48.357 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:vlan database
2026-09-25 19:11:28 10.20.99.12 NOTICE 83: .Sep 25 19:08:48.991 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:vlan database
2026-09-25 19:11:30 10.20.99.11 NOTICE 84: .Sep 25 19:08:50.557 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:vlan 45 name MARKETING
2026-09-25 19:11:31 10.20.99.12 NOTICE 84: .Sep 25 19:08:51.227 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:vlan 45 name MARKETING
2026-09-25 19:11:32 10.20.99.11 NOTICE 85: .Sep 25 19:08:53.933 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:apply
2026-09-25 19:11:33 10.20.99.12 NOTICE 85: .Sep 25 19:08:53.263 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:apply
2026-09-25 19:13:49 10.20.99.11 NOTICE 86: .Sep 25 19:08:24.529 AEST: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: netadmin] [Source: 10.30.100.10] [localport
: 22] at 19:08:24 AEST Fr
2026-09-25 19:13:58 10.20.99.11 NOTICE 87: .Sep 25 19:09:29.249 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:spanning-tree vlan 45 priority 409
6
2026-09-25 19:13:58 10.20.99.11 NOTICE 88: .Sep 25 19:09:29.561 AEST: %SYS-5-CONFIG_I: Configured from console by netadmin on vty0 (10.30.100.10)
2026-09-25 19:14:02 10.20.99.12 NOTICE 86: .Sep 25 19:09:33.669 AEST: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: netadmin] [Source: 10.30.100.10] [localport
: 22] at 19:09:33 AEST Fr
2026-09-25 19:14:25 10.0.0.2 NOTICE 35: Sep 25 19:14:24.903 AEST: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: netadmin] [Source: 10.30.100.10] [localport:
22] at 19:14:24 AEST Fr
2026-09-25 19:14:31 10.0.0.2 NOTICE 36: Sep 25 19:14:30.839 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:interface FastEthernet0/0.45
2026-09-25 19:14:31 10.0.0.2 NOTICE 37: Sep 25 19:14:31.251 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:description VLAN45-MARKETING
2026-09-25 19:14:31 10.0.0.2 NOTICE 38: Sep 25 19:14:31.611 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:encapsulation dot1Q 45
2026-09-25 19:14:33 10.0.0.2 NOTICE 39: Sep 25 19:14:32.123 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:ip address 10.20.45.1 255.255.255.0
2026-09-25 19:14:33 10.0.0.2 NOTICE 40: Sep 25 19:14:32.343 AEST: %SYS-5-CONFIG_I: Configured from console by netadmin on vty0 (10.30.100.10)
2026-09-25 19:14:43 10.0.0.2 NOTICE 41: Sep 25 19:14:42.275 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:router ospf 1
2026-09-25 19:14:43 10.0.0.2 NOTICE 42: Sep 25 19:14:42.723 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:network 10.20.45.0 0.0.0.255 area 2
2026-09-25 19:14:43 10.0.0.2 NOTICE 43: Sep 25 19:14:42.847 AEST: %SYS-5-CONFIG_I: Configured from console by netadmin on vty0 (10.30.100.10)
2026-09-25 19:14:59 10.0.0.2 NOTICE 44: Sep 25 19:14:58.923 AEST: %SEC_LOGIN-5-LOGIN_SUCCESS: Login Success [user: netadmin] [Source: 10.30.100.10] [localport:
22] at 19:14:58 AEST Fr
2026-09-25 19:15:10 10.0.0.2 NOTICE 45: Sep 25 19:15:08.519 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:ip dhcp excluded-address 10.20.45.1
10.20.45.20
2026-09-25 19:15:10 10.0.0.2 NOTICE 46: Sep 25 19:15:08.863 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:ip dhcp pool BR-MARKETING
2026-09-25 19:15:10 10.0.0.2 NOTICE 47: Sep 25 19:15:09.251 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:network 10.20.45.0 255.255.255.0
2026-09-25 19:15:10 10.0.0.2 NOTICE 48: Sep 25 19:15:09.555 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:default-router 10.20.45.1
2026-09-25 19:15:12 10.0.0.2 NOTICE 49: Sep 25 19:15:09.795 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:dns-server 10.0.0.3
2026-09-25 19:15:12 10.0.0.2 NOTICE 50: Sep 25 19:15:10.127 AEST: %PARSER-5-CFGLOG_LOGGEDCMD: User:netadmin logged command:domain-name corp.local
2026-09-25 19:15:12 10.0.0.2 NOTICE 51: Sep 25 19:15:10.391 AEST: %SYS-5-CONFIG_I: Configured from console by netadmin on vty0 (10.30.100.10)

=== STEP COMPLETE ===
((venv)) root@ANSIBLE-SRV:~/ansible#

```

Figure 8: Central Syslog collector receiving messages from all devices

4. Network Automation with Ansible

ANSIBLE-SRV (10.30.100.10) is the control node. It reaches every device over SSH using the `network_cli` connection. The inventory groups devices by role and by site; group and host variables form the single source of truth; secrets are encrypted with Ansible Vault.

The inventory places every device in two kinds of group. Role groups (core routers, site routers, distribution switches and access switches) decide which roles apply, while site groups (HQ, Branch, DC and Core) supply site data such as VLANs, DHCP pools and ACLs. Platform differences are handled in variables rather than in code, for example the older IOS 12.4 switches use DES for SNMPv3 privacy and a shorter change-logging command, so the same roles run unchanged on both router and switch images.

Each role follows the same pattern. Data in `group_vars` and `host_vars` describes what the network should look like, for example the VLAN list for a site or the point-to-point links of a router. Jinja2 templates render that data into Cisco configuration, and the `cisco.ios` modules compare it with the running configuration so that only differences are pushed. A handler saves the configuration to NVRAM only when something actually changed. The same YAML entry therefore feeds several devices at once: a VLAN definition creates the VLAN on both site switches, the router sub-interface, the DHCP pool and the OSPF network statement, which removes the risk of these drifting apart.

Security was built into the automation itself. Device credentials and SNMPv3 keys are held in an AES-256 Ansible Vault file whose password is never committed. The baseline role enforces SSH v2, login brute-force protection, a legal banner, remote logging and configuration-change archiving on every device. The compliance role then checks fourteen security rules against each running configuration and writes a report, so any drift from the standard is detected on the next run.

Playbook / role	Purpose
00 facts	Discover devices and write an inventory CSV
01 backup	Timestamped running-config backups
02 baseline	Hardening, SSH, NTP, Syslog, DNS, SNMPv3, banner, change logging
03 switching	VLANs, trunks, access ports, STP root
04 routing	Sub-interfaces, P2P links, multi-area OSPF (Jinja2)
05 services	DHCP pools, DNS records, NAT (Jinja2)
06 acl	Data-driven, declarative ACLs
07 verify / 08 compliance	Automated assertions and a 14-rule security audit

Table 3: Playbooks and roles

```

=====
LIVE CONSOLE | ANSIBLE-SRV - Ansible control node (10.30.100.10) [192.168.56.104:5001]
=====
>>> Part B | Ansible Vault - device credentials encrypted with AES-256 (values never shown)

((venv) ) root@ANSIBLE-SRV:~/ansible# rm -rf /root/.ansible/collections; ls -l inventory/group_vars/all; echo; head -6 inventory/group_vars/all/vault.yml; echo '...'; echo; echo '# decrypted variable NAMES only: '; ansible-vault view inventory/group_vars/all/vault.yml | sed -E 's/:.*:/: *****/'; echo; grep -n vault_inventory/
group_vars/all/main.yml roles/baseline/tasks/main.yml
total 8
-rw-rw-rw- 1 root root 1392 Sep 25 14:38 main.yml
-rw-rw-rw- 1 root root 1067 Sep 25 15:37 vault.yml

$ANSIBLE_VAULT;1.1;AES256
3365349766313863932338336326530383662663966613562323037366130636332336338646330
646262323864626233642437393736653723337343638370a633665323266383063613366393433
663638386665634373734356536633561316335653064636237633323866646634343838646531
3366313838393231660a6265613765376236326336353034396663736383735636164323838666661
3931383064363965326663362333639333432326461366439393065343566623561323862666430
...

# decrypted variable NAMES only:
---
vault_admin_password: ****
vault_enable_password: ****
vault_ospf_key: ****
vault_snmp_auth: ****
vault_snmp_priv: ****

inventory/group_vars/all/main.yml:6:ansible_password: "{{ vault_admin_password }}"
inventory/group_vars/all/main.yml:9:ansible_become_password: "{{ vault_enable_password }}"
roles/baseline/tasks/main.yml:26:   configured_password: "{{ vault_admin_password }}"
roles/baseline/tasks/main.yml:111:   - snmp-server user {{ snmp.user }} {{ snmp.group }} v3 auth sha {{ vault_snmp_auth }} priv aes 128 {{ vault_snmp_priv }}"
((venv) ) root@ANSIBLE-SRV:~/ansible#

```

Figure 9: Ansible Vault: credentials encrypted with AES-256

```

=====
LIVE CONSOLE | ANSIBLE-SRV - Ansible control node (10.30.100.10) [192.168.56.104:5001]
=====
>>> Part B | 01_backup.yml - back up running-config of every device

((venv) ) root@ANSIBLE-SRV:~/ansible# ansible-playbook playbooks/01_backup.yml 2>&1 | tail -30; ls -l backups/latest/; echo __DEMO_DONE__
ok: [SW4-BR-ACC] =>
msg: SW4-BR-ACC running-config saved to SW4-BR-ACC_20260925-173202.cfg
ok: [SW6-DC-ACC] =>
msg: SW6-DC-ACC running-config saved to SW6-DC-ACC_20260925-173202.cfg

PLAY RECAP *****
R1-HQ      : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
R2-BR      : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
R3-DC      : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
R4-CORE    : ok=5  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
SW1-HQ-DIST : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
SW2-HQ-ACC  : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
SW3-BR-DIST : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
SW4-BR-ACC  : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
SW5-DC-DIST : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
SW6-DC-ACC  : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

TASKS RECAP *****
Friday 25 September 2026  17:32:38 +1000 (0:00:01.476)    0:00:36.626 *****
=====
backup: Back up running-config (R4-CORE) ----- 23.67s
backup: Maintain a 'latest' copy for Git diffing ----- 8.86s
backup: Ensure backup directory exists on the automation server ----- 1.77s
backup: Show backup location ----- 1.48s
backup: Create timestamp for this backup run ----- 0.70s

PLAYBOOK RECAP *****
Playbook run took 0 days, 0 hours, 0 minutes, 36 seconds

total 48
-rw-r--r-- 1 root root 4578 Sep 25 17:32 R1-HQ.cfg
-rw-r--r-- 1 root root 3189 Sep 25 17:32 R2-BR.cfg
-rw-r--r-- 1 root root 4149 Sep 25 17:32 R3-DC.cfg
-rw-r--r-- 1 root root 3303 Sep 25 17:32 R4-CORE.cfg
-rw-r--r-- 1 root root 2812 Sep 25 17:32 SW1-HQ-DIST.cfg
-rw-r--r-- 1 root root 2754 Sep 25 17:32 SW2-HQ-ACC.cfg
-rw-r--r-- 1 root root 2776 Sep 25 17:32 SW3-BR-DIST.cfg
-rw-r--r-- 1 root root 2696 Sep 25 17:32 SW4-BR-ACC.cfg
-rw-r--r-- 1 root root 2777 Sep 25 17:32 SW5-DC-DIST.cfg
-rw-r--r-- 1 root root 2701 Sep 25 17:32 SW6-DC-ACC.cfg
__DEMO_DONE__
((venv) ) root@ANSIBLE-SRV:~/ansible#

```

Figure 10: Backup playbook saving all ten running configurations

```

LIVE - ANSIBLE-SRV (Ansible) x + v
((venv) ) root@ANSIBLE-SRV:~/ansible# ansible-playbook playbooks/07_verify.yml 2>&1 | tail -45; echo === STEP COMPLETE ===
Friday 25 September 2026  18:30:46 +1000 (0:00:13.873)          0:01:11.194 *****
ok: [R1-HQ]
ok: [R4-CORE]
ok: [R2-BR]
ok: [SW4-BR-ACC]
ok: [SW2-HQ-ACC]
ok: [SW5-DC-DIST]
ok: [R3-DC]
ok: [SW1-HQ-DIST]
ok: [SW3-BR-DIST]
ok: [SW6-DC-ACC]

TASK [verify : Write verification report] *****
Friday 25 September 2026  18:30:47 +1000 (0:00:01.562)          0:01:12.757 *****
changed: [R4-CORE -> localhost]

PLAY RECAP *****
R1-HQ                : ok=5    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
R2-BR                : ok=5    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
R3-DC                : ok=5    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
R4-CORE              : ok=6    changed=1    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
SW1-HQ-DIST          : ok=4    changed=0    unreachable=0    failed=0    skipped=4    rescued=0    ignored=0
SW2-HQ-ACC           : ok=4    changed=0    unreachable=0    failed=0    skipped=4    rescued=0    ignored=0
SW3-BR-DIST          : ok=4    changed=0    unreachable=0    failed=0    skipped=4    rescued=0    ignored=0
SW4-BR-ACC           : ok=4    changed=0    unreachable=0    failed=0    skipped=4    rescued=0    ignored=0
SW5-DC-DIST          : ok=4    changed=0    unreachable=0    failed=0    skipped=4    rescued=0    ignored=0
SW6-DC-ACC           : ok=4    changed=0    unreachable=0    failed=0    skipped=4    rescued=0    ignored=0

TASKS RECAP *****
Friday 25 September 2026  18:30:52 +1000 (0:00:05.013)          0:01:17.771 *****
=====
verify : Reachability tests from each router (DC web server, NetOps server, Internet) -- 23.27s
verify : Collect operational state (routers) ----- 16.13s
verify : Collect operational state (switches) ----- 14.95s
verify : Switch reachability to its gateway and the NTP master ----- 13.87s
verify : Write verification report ----- 5.01s
verify : Record NTP sync state ----- 1.56s
verify : Assert all site VLANs exist and trunks are up ----- 1.35s
verify : Assert default route learned / originated ----- 1.08s
verify : Assert every OSPF adjacency is FULL ----- 0.47s

PLAYBOOK RECAP *****
Playbook run took 0 days, 0 hours, 1 minutes, 17 seconds

=== STEP COMPLETE ===
((venv) ) root@ANSIBLE-SRV:~/ansible#

```

Figure 11: Verification playbook: OSPF, routes, VLANs, trunks and reachability pass

```

LIVE - ANSIBLE-SRV (Ansible) x + v
((venv) ) root@ANSIBLE-SRV:~/ansible# ansible-playbook playbooks/08_compliance.yml 2>&1 | grep -E 'msg:|failed=' | tail -24; echo; cat reports/compliance_report.md;
echo === STEP COMPLETE ===
msg: R4-CORE compliance 14/14 ALL PASS
msg: R2-BR compliance 14/14 ALL PASS
msg: R1-HQ compliance 14/14 ALL PASS
msg: R3-DC compliance 14/14 ALL PASS
msg: SW4-BR-DIST compliance 14/14 ALL PASS
msg: SW1-HQ-DIST compliance 14/14 ALL PASS
msg: SW5-DC-DIST compliance 14/14 ALL PASS
msg: SW4-BR-ACC compliance 14/14 ALL PASS
msg: SW2-HQ-ACC compliance 14/14 ALL PASS
msg: SW6-DC-ACC compliance 14/14 ALL PASS
R1-HQ                : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
R2-BR                : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
R3-DC                : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
R4-CORE              : ok=5    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW1-HQ-DIST          : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW2-HQ-ACC           : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW3-BR-DIST          : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW4-BR-ACC           : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW5-DC-DIST          : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW6-DC-ACC           : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

# ENT-HYBRID-NET - Security Compliance Report
Generated by Ansible at 2026-09-25 18:37:25 AEST

| Rule | R1-HQ | R2-BR | R3-DC | R4-CORE | SW1-HQ-DIST | SW2-HQ-ACC | SW3-BR-DIST | SW4-BR-ACC | SW5-DC-DIST | SW6-DC-ACC |
|---|---|---|---|---|---|---|---|---|---|---|
SSH version 2 only | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
Telnet disabled on VTY | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
VTY protected by SSH | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
Local AAA user | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
Enable secret (not password) | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
Password encryption | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
Login brute-force protection | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
Legal banner | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
NTP configured | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
Remote syslog | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
Config-change archive logging | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
SNMPv3 priv group | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
No SNMPv1/v2c communities | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
HTTP server disabled | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS | PASS |
**Score** | 14/14 | 14/14 | 14/14 | 14/14 | 14/14 | 14/14 | 14/14 | 14/14 | 14/14 | 14/14 |

=== STEP COMPLETE ===
((venv) ) root@ANSIBLE-SRV:~/ansible#

```

Figure 12: Compliance audit: 14 of 14 rules pass on every device

```

=====
LIVE CONSOLE | ANSIBLE-SRV - Ansible control node (10.30.100.10) [192.168.56.104:5001]
=====
>>> Part B | Idempotency: full configuration pipeline (02-06) second run - PLAY RECAP
=====
((venv) ) root@ANSIBLE-SRV:~/ansible# grep -a -B2 -A11 'PLAY RECAP' logs/ansible.log | tail -13 | sed 's/^.*n=ansible | ///'; echo; echo 'Playbook run took 6 minutes
24 seconds: 5 playbooks, 10 devices, changed=0 unreachable=0 failed=0'; echo === STEP COMPLETE ===
=====
PLAY RECAP *****
R1-HQ      : ok=18  changed=0  unreachable=0  failed=0  skipped=1  rescued=0  ignored=0
R2-BR      : ok=17  changed=0  unreachable=0  failed=0  skipped=2  rescued=0  ignored=0
R3-DC      : ok=17  changed=0  unreachable=0  failed=0  skipped=2  rescued=0  ignored=0
R4-CORE    : ok=17  changed=0  unreachable=0  failed=0  skipped=2  rescued=0  ignored=0
SW1-HQ-DIST : ok=17  changed=0  unreachable=0  failed=0  skipped=3  rescued=0  ignored=0
SW2-HQ-ACC  : ok=17  changed=0  unreachable=0  failed=0  skipped=3  rescued=0  ignored=0
SW3-BR-DIST : ok=17  changed=0  unreachable=0  failed=0  skipped=3  rescued=0  ignored=0
SW4-BR-ACC  : ok=17  changed=0  unreachable=0  failed=0  skipped=3  rescued=0  ignored=0
SW5-DC-DIST : ok=17  changed=0  unreachable=0  failed=0  skipped=3  rescued=0  ignored=0
SW6-DC-ACC  : ok=17  changed=0  unreachable=0  failed=0  skipped=3  rescued=0  ignored=0
TASKS RECAP *****
=====
Playbook run took 6 minutes 24 seconds: 5 playbooks, 10 devices, changed=0 unreachable=0 failed=0
=== STEP COMPLETE ===
((venv) ) root@ANSIBLE-SRV:~/ansible#

```

Figure 13: Idempotency: second run of the whole pipeline reports changed=0

The verification role asserts that every OSPF adjacency is FULL, that routers learn the default route, that each switch has its VLANs and trunks, and that devices can reach the Data Centre servers and the Internet. Because every check is an assertion, a broken network makes the playbook fail rather than silently succeed. The results are also written to a Markdown report that is committed to Git as evidence.

Backups and Git work together. The backup playbook saves a timestamped copy of every running configuration and keeps a latest copy per device in the repository, so any difference between two runs appears as a normal Git diff. Before committing, reversible type-7 strings are redacted, and a scan of the full history confirmed that no plain-text secret was ever pushed. This gives the team an auditable record of how the network changed over time.

4.1 Day-2 change through Git

To add VLAN 45 MARKETING at the Branch, we changed two lines of YAML and committed them to Git. A --check --diff dry run previewed the change, the pipeline applied it, and the devices confirmed the new VLAN, gateway sub-interface, DHCP pool and OSPF network. No CLI was typed by hand.

```

=====
LIVE CONSOLE | ANSIBLE-SRV - Ansible control node (10.30.100.10) [192.168.56.104:5001]
=====
>>> Part B | Day-2 CHG-0001 dry run: --check --diff on site_branch (nothing is applied)
=====
((venv) ) root@ANSIBLE-SRV:~/ansible# ansible-playbook playbooks/03_switching.yml playbooks/04_routing.yml playbooks/05_services.yml -l site_branch --check --diff 2>
&1 | grep -E 'PLAY \[changed:|would create|'failed=' | tail -46; echo === STEP COMPLETE ===
=====
PLAY [Part B-3 | Switching - VLANs, 802.1Q trunks, access ports, STP root] *****
changed: [SW4-BR-ACC] =>
  msg: 'would create: ['45 MARKETING']'
changed: [SW3-BR-DIST] =>
  msg: 'would create: ['45 MARKETING']'
changed: [SW3-BR-DIST]
SW3-BR-DIST : ok=6  changed=2  unreachable=0  failed=0  skipped=3  rescued=0  ignored=0
SW4-BR-ACC  : ok=6  changed=1  unreachable=0  failed=0  skipped=2  rescued=0  ignored=0
PLAY [Part B-4 | Routing - loopbacks, inter-VLAN sub-interfaces, P2P links, multi-area OSPF] ***
changed: [R2-BR]
R2-BR      : ok=2  changed=2  unreachable=0  failed=0  skipped=1  rescued=0  ignored=0
SW3-BR-DIST : ok=6  changed=2  unreachable=0  failed=0  skipped=3  rescued=0  ignored=0
SW4-BR-ACC  : ok=6  changed=1  unreachable=0  failed=0  skipped=2  rescued=0  ignored=0
PLAY [Part B-5 | Network services - DHCP pools, DNS server records, NAT/PAT] ***
changed: [R2-BR]
R2-BR      : ok=3  changed=3  unreachable=0  failed=0  skipped=2  rescued=0  ignored=0
SW3-BR-DIST : ok=6  changed=2  unreachable=0  failed=0  skipped=3  rescued=0  ignored=0
SW4-BR-ACC  : ok=6  changed=1  unreachable=0  failed=0  skipped=2  rescued=0  ignored=0
===== STEP COMPLETE =====
((venv) ) root@ANSIBLE-SRV:~/ansible#

```

Figure 14: Dry run predicts the change without touching devices

```

Part B | Day-2 CHG-0001 verified on R2-BR - Fa0/0.45 gateway, DHCP pool BR-MARKETING, OSPF area 20 [localhost:5003] 2026-09-25 19:15:32
=====
R2-BR#show ip interface brief | include 0/0. | Loopback
FastEthernet0/0      unassigned      YES NVRAM  up            up
FastEthernet0/0.40   10.20.40.1      YES NVRAM  up            up
FastEthernet0/0.45   10.20.45.1      YES manual up            up
FastEthernet0/0.50   10.20.50.1      YES NVRAM  up            up
FastEthernet0/0.99   10.20.99.1      YES NVRAM  up            up
Loopback0            10.0.0.2        YES NVRAM  up            up
R2-BR#show running-config interface FastEthernet0/0.45
Building configuration...

Current configuration : 127 bytes
!
interface FastEthernet0/0.45
 description VLAN45-MARKETING
 encapsulation dot1Q 45
 ip address 10.20.45.1 255.255.255.0
 end

R2-BR#show ip dhcp pool BR-MARKETING
Pool BR-MARKETING :
Utilization mark (high/low)    : 100 / 0
Subnet size (first/next)       : 0 / 0
Total addresses                 : 254
Leased addresses               : 0
Pending event                   : none
1 subnet is currently in the pool :
Current index      IP address range      Leased addresses
10.20.45.1         10.20.45.1 - 10.20.45.254    0
R2-BR#show ip ospf interface brief | include 0.45
Fa0/0.45          1      20      10.20.45.1/24      10      DR      0/0
R2-BR#
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation>

```

Figure 15: R2-BR after the change: new gateway, DHCP pool and OSPF interface

4.2 Infrastructure as Code discussion

Infrastructure as Code manages infrastructure through versioned, machine-readable definitions rather than manual commands [5], [6]. Here the network's desired state lives in YAML, templates turn it into configuration, and Ansible makes devices match. Runs are idempotent, so they are safe to repeat and correct drift. Every change is a reviewed Git commit that can be rolled back, and the verify and compliance roles test the result automatically.

5. AWS Infrastructure as Code with Terraform

ENT-HYBRID-NET – AWS Architecture (Terraform, ap-southeast-2)

VPC 172.20.0.0/16 · 2 AZs · 2 public + 2 private subnets · IGW · route tables · security groups · bastion + private EC2 · SSH key pair

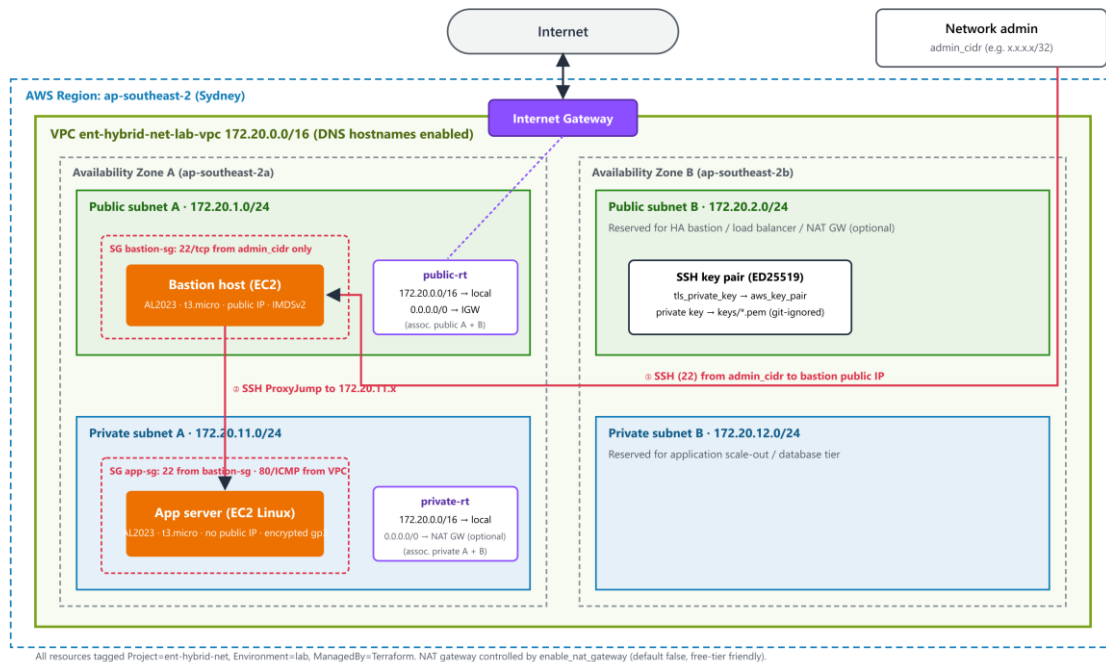


Figure 16: AWS architecture provisioned by Terraform

Terraform creates a VPC (172.20.0.0/16) with two public and two private subnets across two Availability Zones, an Internet Gateway, public and private route tables, security groups, an ED25519 key pair, a bastion host in a public subnet and an Amazon Linux EC2 instance in a private subnet. The bastion accepts SSH only from the administrator's /32 address; the private server accepts SSH only from the bastion's security group. IMDSv2 and encrypted volumes are enforced, and state, keys and variable files are excluded from Git.

Terraform is declarative: it compares the configuration with its state and the real account, then plans the required actions [2]. Our workflow was `init`, `fmt`, `validate`, `plan -out tfplan`, `apply tfplan`, `verification` and `destroy`. The code was committed in stages (providers, network, security, compute, outputs) to show version control.

The code is organised by concern into `versions.tf`, `variables.tf`, `network.tf`, `security.tf`, `compute.tf` and `outputs.tf`. Input variables carry validation rules, for example refusing 0.0.0.0/0 as the administrator range, and default tags label every resource with the project and `ManagedBy=Terraform`. A NAT gateway is available through a variable but disabled by default to stay within the free tier, which is why the private server intentionally has no Internet access. The provider versions are pinned in the committed lock file so every engineer gets identical results.

The saved plan was applied, creating all 25 resources in about thirty seconds. The deployment was then checked independently with the AWS CLI and with a real SSH session through the bastion to the private server. Finally `terraform destroy` removed every resource, and an audit of all AWS regions confirmed that no instances, volumes, Elastic IPs, NAT gateways, custom VPCs, key pairs or snapshots remained, so the environment incurs no ongoing cost.

Security follows AWS Well-Architected guidance [4]. Only the bastion is reachable from the Internet, and only on port 22 from one address. The application server has no public IP and sits behind a security group that trusts the bastion group, not an IP range, so the rule stays correct if the bastion is replaced. The private key is generated by Terraform, written with restricted permissions and excluded from Git, and instance metadata requires session tokens, which blocks a common credential-theft technique.

```
Part C Terraform plan
>>> Part C | terraform plan -out tfplan (AWS account via profile terraform-user, region ap-southeast-2)
# aws_instance.app will be created
# aws_instance.bastion will be created
# aws_internet_gateway.igw will be created
# aws_key_pair.admin will be created
# aws_route_table.private will be created
# aws_route_table.public will be created
# (12 unchanged attributes hidden)
# aws_route_table_association.private[0] will be created
# aws_route_table_association.private[1] will be created
# aws_route_table_association.public[0] will be created
# aws_route_table_association.public[1] will be created
# aws_security_group.app will be created
# aws_security_group.bastion will be created
# aws_subnet.private[0] will be created
# aws_subnet.private[1] will be created
# aws_subnet.public[0] will be created
# aws_subnet.public[1] will be created
# aws_vpc.main will be created
# aws_vpc_security_group_egress_rule.app_all will be created
# aws_vpc_security_group_egress_rule.bastion_all will be created
# aws_vpc_security_group_ingress_rule.app_http_from_vpc will be created
# aws_vpc_security_group_ingress_rule.app_icmp_from_vpc will be created
# aws_vpc_security_group_ingress_rule.app_ssh_from_bastion will be created
# aws_vpc_security_group_ingress_rule.bastion_ssh will be created
# local_sensitive_file.private_key will be created
# tls_private_key.ssh will be created
Plan: 25 to add, 0 to change, 0 to destroy.

Plan: 25 to add, 0 to change, 0 to destroy.
Changes to Outputs:
Saved the plan to: tfplan
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation\terraform> |
```

Figure 17: terraform plan: 25 resources to add

```

Part C Terraform apply
>>> Part C | terraform plan -out tfplan ; terraform apply tfplan (ap-southeast-2)
Plan: 25 to add, 0 to change, 0 to destroy.

tls_private_key.ssh: Creation complete after 0s [id=aac4d841173752cf857e25c17a89c83ed15bb3be]
local_sensitive_file.private_key: Creation complete after 0s [id=ent-hybrid-net-lab-key]
aws_vpc.main: Creation complete after 2s [id=vpc-0e325d2e5ff2e7bec]
aws_internet_gateway.igw: Creation complete after 0s [id=igw-04d7b5af547e848c0]
aws_route_table.private: Creation complete after 0s [id=rtb-0d3b86b5c8f34019f]
aws_subnet.private[0]: Creation complete after 0s [id=subnet-0066ed10cd95be246]
aws_subnet.private[1]: Creation complete after 0s [id=subnet-0f52e342968a26a59]
aws_route_table_association.private[1]: Creation complete after 1s [id=rtbassoc-03a25099ad8fe7e32]
aws_route_table_association.private[0]: Creation complete after 1s [id=rtbassoc-0dd5ac232811d4b6a]
aws_route_table.public: Creation complete after 1s [id=rtb-08ca6a96995b83ed0]
aws_security_group.app: Creation complete after 1s [id=sg-0ac51fc4f88cd1875]
aws_security_group.bastion: Creation complete after 1s [id=sg-0985cbd1e130f9c03]
aws_vpc_security_group_ingress_rule.app_icmp_from_vpc: Creation complete after 1s [id=sgr-0c5256f9c50bdec11]
aws_vpc_security_group_egress_rule.bastion_all: Creation complete after 1s [id=sgr-0439a11af381ea8da]
aws_vpc_security_group_egress_rule.app_all: Creation complete after 1s [id=sgr-0cf74147b83ac04d6]
aws_vpc_security_group_ingress_rule.app_http_from_vpc: Creation complete after 1s [id=sgr-06bec07464c8ada3d]
aws_vpc_security_group_ingress_rule.bastion_ssh: Creation complete after 1s [id=sgr-09b07df4a7bb18230]
aws_vpc_security_group_ingress_rule.app_ssh_from_bastion: Creation complete after 1s [id=sgr-0c91e44d533d45609]
aws_subnet.public[1]: Creation complete after 11s [id=subnet-09a83c336ab716a75]
aws_subnet.public[0]: Creation complete after 11s [id=subnet-04004dd0dacedd767b]
aws_route_table_association.public[1]: Creation complete after 0s [id=rtbassoc-000cacbcaebd18b56]
aws_route_table_association.public[0]: Creation complete after 0s [id=rtbassoc-0733f613e4123cda0]
aws_instance.app: Creation complete after 13s [id=i-01681336ed920bcfa]
aws_instance.bastion: Creation complete after 13s [id=i-0480dd1b0199531c3]
Apply complete! Resources: 25 added, 0 changed, 0 destroyed.
Outputs:
app_private_ip = "172.20.11.109"
bastion_public_ip = "3.107.193.253"
internet_gateway_id = "igw-04d7b5af547e848c0"
private_subnet_ids = [
public_subnet_ids = [
ssh_to_app_via_bastion = "ssh -i keys/ent-hybrid-net-lab-key.pem -J ec2-user@3.107.193.253 ec2-user@172.20.11.109"
ssh_to_bastion = "ssh -i keys/ent-hybrid-net-lab-key.pem ec2-user@3.107.193.253"
vpc_id = "vpc-0e325d2e5ff2e7bec"
=== APPLY STEP DONE ===

PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation\terraform>

```

Figure 18: terraform apply: 25 resources created, with outputs

```

Part C AWS CLI verification
>>> Part C | AWS CLI - resources created by Terraform (tag Project:ent-hybrid-net, ap-southeast-2)

DescribeVpcs
+-----+-----+-----+
| CIDR | Name | VPC |
+-----+-----+-----+
| 172.20.0.0/16 | ent-hybrid-net-lab-vpc | vpc-0e325d2e5ff2e7bec |
+-----+-----+-----+

DescribeSubnets
+-----+-----+-----+-----+-----+
| AZ | CIDR | Name | PublicIP | Subnet |
+-----+-----+-----+-----+-----+
| ap-southeast-2b | 172.20.12.0/24 | ent-hybrid-net-lab-private-ap-southeast-2b | False | subnet-0f52e342968a26a59 |
| ap-southeast-2a | 172.20.11.0/24 | ent-hybrid-net-lab-private-ap-southeast-2a | False | subnet-0066ed10cd95be246 |
| ap-southeast-2a | 172.20.1.0/24 | ent-hybrid-net-lab-public-ap-southeast-2a | True | subnet-04004dd0dacedd767b |
| ap-southeast-2b | 172.20.2.0/24 | ent-hybrid-net-lab-public-ap-southeast-2b | True | subnet-09a83c336ab716a75 |
+-----+-----+-----+-----+-----+

aws: [ERROR]: An error occurred (ParamValidation): Bad value for --query RouteTables[0].RT:RouteTableId,Name:Tags[?Key=='Name']|[0].Value,Routes:join(, ,Routes[0].join(, ,DestinationCidrBlock,GatewayId))]: Bad jmespath expression: Unknown token '-'
RouteTables[0].{RT:RouteTableId,Name:Tags[?Key=='Name']|[0].Value,Routes:join(, ,Routes[0].join(, ,DestinationCidrBlock,GatewayId))}

DescribeInstances
+-----+-----+-----+-----+-----+-----+
| Instance | Name | PrivateIP | PublicIP | State | Subnet | Type |
+-----+-----+-----+-----+-----+-----+
| i-01681336ed920bcfa | ent-hybrid-net-lab-app | 172.20.11.109 | None | running | subnet-0066ed10cd95be246 | t3.micro |
| i-0480dd1b0199531c3 | ent-hybrid-net-lab-bastion | 172.20.1.231 | 3.107.193.253 | running | subnet-04004dd0dacedd767b | t3.micro |
+-----+-----+-----+-----+-----+-----+

aws: [ERROR]: An error occurred (ParamValidation): Bad value for --query SecurityGroups[0].SG:GroupId,Name:GroupName,Ingress:join(, ,IpPermissions[0].join(, ,[IpProtocol,to_string(ToPort)])): Bad jmespath expression: Unknown token '/'
SecurityGroups[0].{SG:GroupId,Name:GroupName,Ingress:join(, ,IpPermissions[0].join(, ,[IpProtocol,to_string(ToPort)]))}

PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation>

```

Figure 19: AWS CLI confirms VPC, subnets and running instances

```

Part C SSH bastion ProxyJump
>>> Part C | SSH to bastion (public subnet), then ProxyJump to the private EC2 instance (no public IP)
$ ssh -i keys/ent-hybrid-net-lab-key.pem ec2-user@3.107.193.253
host: bastion
2: ens5      inet 172.20.1.231/24 metric 512 brd 172.20.1.255 scope global dynamic ens5\        valid_lft 2899sec preferred_lft 2899sec
Internet via IGW: HTTP 200

$ ssh -i keys/ent-hybrid-net-lab-key.pem -J ec2-user@3.107.193.253 ec2-user@172.20.11.109
host: app-server
2: ens5      inet 172.20.11.109/24 metric 512 brd 172.20.11.255 scope global dynamic ens5\        valid_lft 2888sec preferred_lft 2888sec
2 packets transmitted, 0 received, 100% packet loss, time 1009ms

Internet: NOT reachable (private subnet, no NAT gateway - as designed)
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation\terraform>

```

Figure 20: SSH to the bastion and ProxyJump to the private server

```

Part C AWS post-destroy billir x + v
>>> Part C | Post-destroy billing audit of AWS account (all regions) - independent of Terraform state

Region      Instances EBSVolumes ElasticIPs NATGateways CustomVPCs KeyPairs Snapshots Status
-----
ap-south-1  0         0         0         0         0         0         0 clean
eu-north-1  0         0         0         0         0         0         0 clean
eu-west-3   0         0         0         0         0         0         0 clean
eu-west-2   0         0         0         0         0         0         0 clean
eu-west-1   0         0         0         0         0         0         0 clean
ap-northeast-3 0         0         0         0         0         0         0 clean
ap-northeast-2 0         0         0         0         0         0         0 clean
ap-northeast-1 0         0         0         0         0         0         0 clean
ca-central-1 0         0         0         0         0         0         0 clean
sa-east-1   0         0         0         0         0         0         0 clean
ap-southeast-1 0         0         0         0         0         0         0 clean
ap-southeast-2 0         0         0         0         0         0         0 clean
eu-central-1 0         0         0         0         0         0         0 clean
us-east-1   0         0         0         0         0         0         0 clean
us-east-2   0         0         0         0         0         0         0 clean
us-west-1   0         0         0         0         0         0         0 clean
us-west-2   0         0         0         0         0         0         0 clean

# ap-southeast-2 detail: terminated instances (not billed) and remaining VPCs
-----
|                                     DescribeInstances                                     |
+-----+-----+-----+-----+-----+-----+-----+-----+
| i-01681336ed920bcfa | terminated | ent-hybrid-net-lab-app |
| i-0480dd1b0199531c3 | terminated | ent-hybrid-net-lab-bastion |
+-----+-----+-----+-----+-----+-----+-----+
|                                     DescribeVpcs                                     |
+-----+-----+-----+-----+-----+-----+
| vpc-02423d81ad9f0be5c | 172.31.0.0/16 | True |
+-----+-----+-----+-----+-----+-----+

RESULT: 0 billable resources in any region - account is clean.
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation>

```

Figure 21: After destroy: no billable resources in any region

```

Part C Git version control hist x + v
>>> git log --graph --oneline --date=short (version control of the whole project)
* ff49f90 2026-09-25 Evidence: Part A/B/C screenshots, Terraform plan output, lab recovery and demo tooling <Shakil Ahammed>
* 6dea959 2026-09-25 Part B: automation outputs - config backups, inventory, verification and compliance reports, syslog <Shakil Ahammed>
* 4cd4e9f 2026-09-25 Part B: make every role idempotent and check-mode safe on IOS 12.4/15.0 <Shakil Ahammed>
* f4fa596 2026-09-25 CHG-0001: add VLAN 45 MARKETING at the Branch Office <Shakil Ahammed>
* 1f6aa70 2026-09-25 Add placeholder B04 backup screenshot (run still in progress) <Shakil Ahammed>
* 56017ad 2026-09-25 Lab stability fixes, offline Ansible install and live demo tooling <Shakil Ahammed>
* 5c0485c9 2026-09-25 Part B: switch control node to ansible-core 2.17 plus required collections <Shakil Ahammed>
* 5d8cb0f 2026-09-25 Part B: Ansible project (inventory, roles, playbooks, server tools) and report generator <Shakil Ahammed>
* b0a96f7 2026-09-25 Part A: evidence screenshots - OSPF, DHCP, ACLs, DNS, VLANs, STP, SSH/NTP/Syslog, host tests <Shakil Ahammed>
* cb05eee 2026-09-25 Tooling: evidence capture (window-only screenshots), clock sync, Ansible sync to automation server <Shakil Ahammed>
* 16eaa78 2026-09-25 Docs: IP addressing plan, design justification, AWS architecture diagram <Shakil Ahammed>
* 8667924 2026-09-25 Part C: Terraform outputs incl. ProxyJump SSH command <Shakil Ahammed>
* c86c4c4 2026-09-25 Part C: SSH key pair, bastion host and private EC2 Linux instance (IMDSv2, encrypted EBS) <Shakil Ahammed>
* 79bda15 2026-09-25 Part C: security groups - bastion SSH from admin CIDR, app SSH from bastion SG only <Shakil Ahammed>
* 43180a1 2026-09-25 Part C: VPC, 2 public + 2 private subnets, IGW, route tables, optional NAT GW <Shakil Ahammed>
* ee729e8 2026-09-25 Part C: Terraform providers, variables with validation, default tags <Shakil Ahammed>
* ceec71 2026-09-25 Part A: console bootstrap tooling (telnet client, parallel config push) <Shakil Ahammed>
* eb2dc73 2026-09-25 Part A: Day-0 config generator and rendered device configs (VLAN, OSPF, DHCP, SSH, ACL, NAT, DNS, NTP, Syslog) <Shakil Ahammed>
* b9fce75 2026-09-25 Part A: move end hosts to GNS3 VM, replace VPCS with Docker Linux hosts (DHCP fix) <Shakil Ahammed>
* bf9cddf 2026-09-25 Part A: GNS3 REST API client and hierarchical topology builder <Shakil Ahammed>
* 1624c4f 2026-09-25 Initial project structure, README and .gitignore <Shakil Ahammed>

commits: 21 branch: main tracked files: 163
PS C:\Users\SHAHAN\Desktop\Enterprise-Network-Automation>

```

Figure 22: Git commit history of the project

6. Testing

Testing combined three methods. Device commands verified routing, switching and services directly on the routers and switches. End-host tests from the Linux containers checked what users actually experience: address assignment, name resolution, reachability and the effect of each ACL. Finally, the Ansible verify and compliance roles repeated the key checks automatically after every change. Table 4 summarises the results, and every test is backed by a screenshot in the evidence folder.

Test	Result
OSPF adjacencies and site summaries	Pass
Internet access through NAT	Pass
DHCP leasing and DNS resolution	Pass
Inter-site and inter-VLAN reachability	Pass
Guest, Finance and server ACLs	Pass
STP redundancy	Pass
SSH-only management and banner	Pass
Central Syslog collection	Pass
NTP	Partial: reachable, sync prevented by emulator clock drift
Ansible idempotency, verification and compliance	Pass
Day-2 change through Git	Pass
Terraform apply, AWS verification and bastion SSH	Pass
Terraform destroy and cost audit	Pass

Table 4: Test results

7. Discussion

- **End hosts:** VPCS could not complete DHCP with Cisco IOS, so the PCs were replaced with small Linux containers, which also provided real test tools and a web server.
- **Emulation limits:** VirtualBox ran the GNS3 VM through Hyper-V, causing periodic freezes; a recovery script restored the lab in minutes. Clock drift between emulated routers prevented NTP synchronisation, although the design and configuration were verified.
- **Idempotency on real IOS:** IOS hides default commands and rewrites others, so roles were adjusted to push exactly what IOS stores. Afterwards repeated runs made no changes.
- **Security of evidence:** secrets were kept in Vault and git-ignored files, and reversible type-7 strings were redacted from committed backups.
- **Offline installation:** large downloads through the emulated Internet path timed out, so Ansible and its collections were installed from packages downloaded on the host. This mirrors how automation is deployed in air-gapped networks.
- **Lessons learned:** automation made mistakes visible early. A malformed variable file was rejected before any device changed, and the dry-run mode let each change be reviewed before it was applied. Writing tests as part of the automation, rather than as a separate manual step, gave confidence that the network still worked after every run.
- **Future work:** a CI/CD pipeline with linting, NetBox as the source of truth, a site-to-site VPN to the AWS VPC and remote Terraform state with locking.

8. Conclusion

The project met its objectives. A hierarchical, secure network was built and verified; Ansible now manages it as code with idempotent, audited and version-controlled changes; and Terraform delivers a secure AWS environment that was deployed, tested and cleanly removed. Together these show the move from manual administration to automated, repeatable hybrid-cloud operations. The main limitations came from the emulated lab rather than the design, and the recommended next steps would bring the same practices into a continuous-delivery pipeline for production.

References

- [1] Red Hat, Inc., “Ansible cisco.ios collection documentation,” 2025. [Online]. Available: <https://docs.ansible.com/ansible/latest/collections/cisco/ios/>
- [2] HashiCorp, “Terraform language documentation,” 2025. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>
- [3] Amazon Web Services, “Amazon VPC user guide,” 2025. [Online]. Available: <https://docs.aws.amazon.com/vpc/>
- [4] Amazon Web Services, “AWS Well-Architected Framework: Security pillar,” 2024.
- [5] K. Morris, Infrastructure as Code, 2nd ed. Sebastopol, CA, USA: O’Reilly, 2020.
- [6] M. Oswalt, C. Adell, S. S. Lowe and J. Edelman, Network Programmability and Automation, 2nd ed. Sebastopol, CA, USA: O’Reilly, 2023.
- [7] Cisco Systems, “IP routing: OSPF configuration guide, IOS 15M&T,” 2023.
- [8] NIST, “Security and privacy controls for information systems,” SP 800-53 Rev. 5, 2020.

Appendix: Submission Contents

Item	Folder
Technical report	01_Report
GNS3 project	02_GNS3_Project
Configuration files	03_Configuration_Files
Ansible project	04_Ansible_Project
Terraform project	05_Terraform_Project
Git repository link	06_Git_Repository_Link.txt
Evidence screenshots	07_Evidence_Screenshots